



BITS Pilani
Pilani Campus

Designing Reliable Microservices

Prof. Akanksha Bharadwaj
Asst. Professor, CSIS Department



BITS Pilani
Pilani Campus



SE ZG583, Scalable Services

Lecture No. 10

Agenda



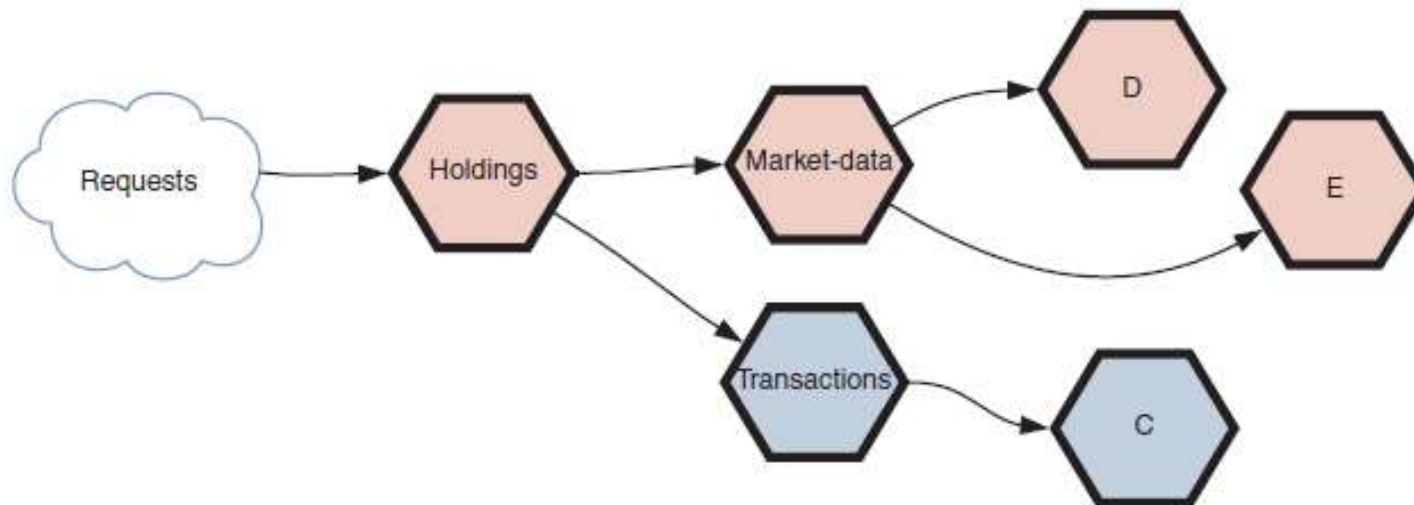
Designing reliable Microservices

- Sources of failure, cascading failures
- Designing reliable communication: Retires, async. Comm., circuit breakers
- Maximizing service reliability: Load balancing, Rate limiting (Queues, Throttling)
- Service mesh

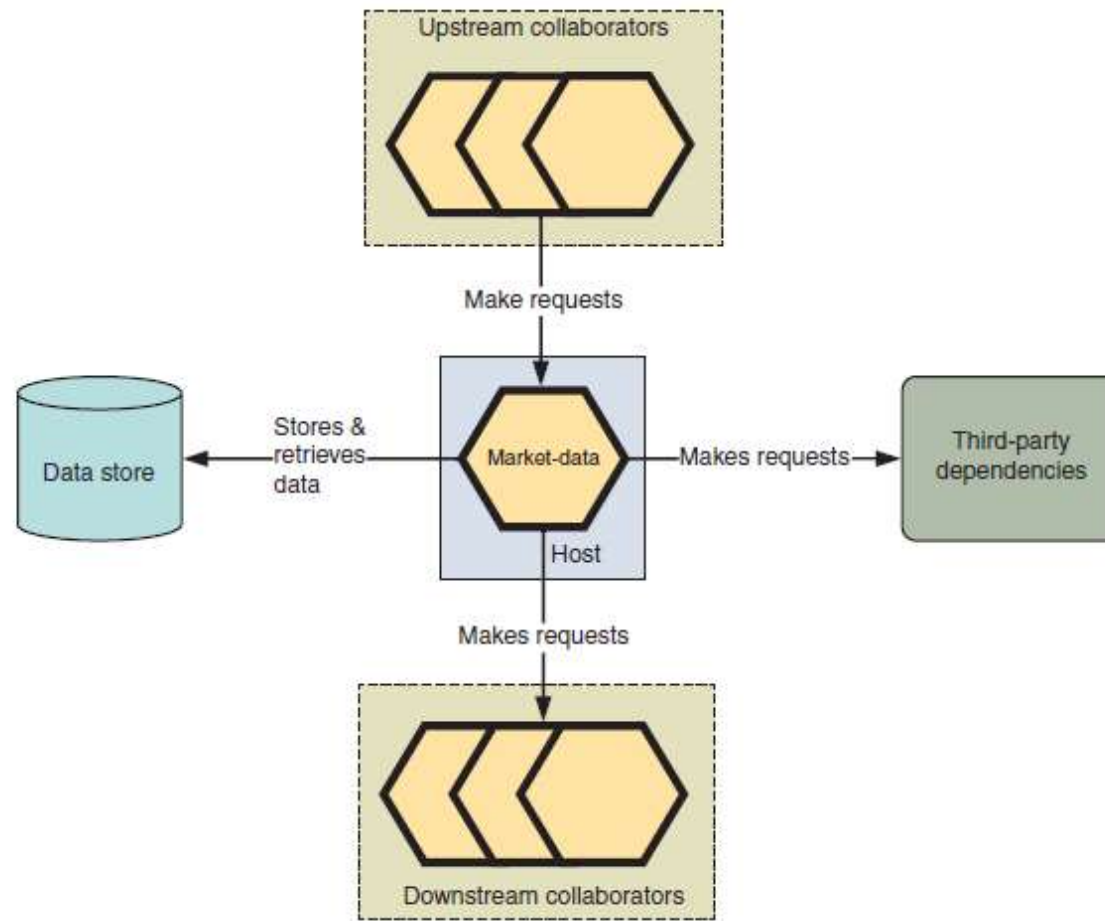
What is reliability?



- A service's availability is a measure of how reliable you can expect it to be.



Example



Sources of failure



Every point of interaction between your service and another component indicates a possible area of failure. Failures could occur in four major areas:

- *Hardware*
- *Communication*
- *Dependencies*
- *Internal*

Hardware Failure



Source of failure	Frequency	Description
Host	Often	Individual hosts (physical or virtual) may fail.
Data center	Rare	Data centers or components within them may fail.
Host configuration	Occasionally	Hosts may be misconfigured — for example, through errors in provisioning tools.
Physical network	Rare	Physical networking (within or between data centers) may fail.
Operating system and resource isolation	Occasionally	The OS or the isolation system — for example, Docker — may fail to operate correctly.

Communication Failure



Source of failure	Description
Network	Network connectivity may not be possible.
Firewall	Configuration management can set security rules inappropriately.
DNS errors	Hostnames may not be correctly propagated or resolved across an application.
Messaging	Messaging systems—for example, RPC—can fail.
Inadequate health checks	Health checks may not adequately represent instance state, causing requests to be routed to broken instances.

Dependency-related failure



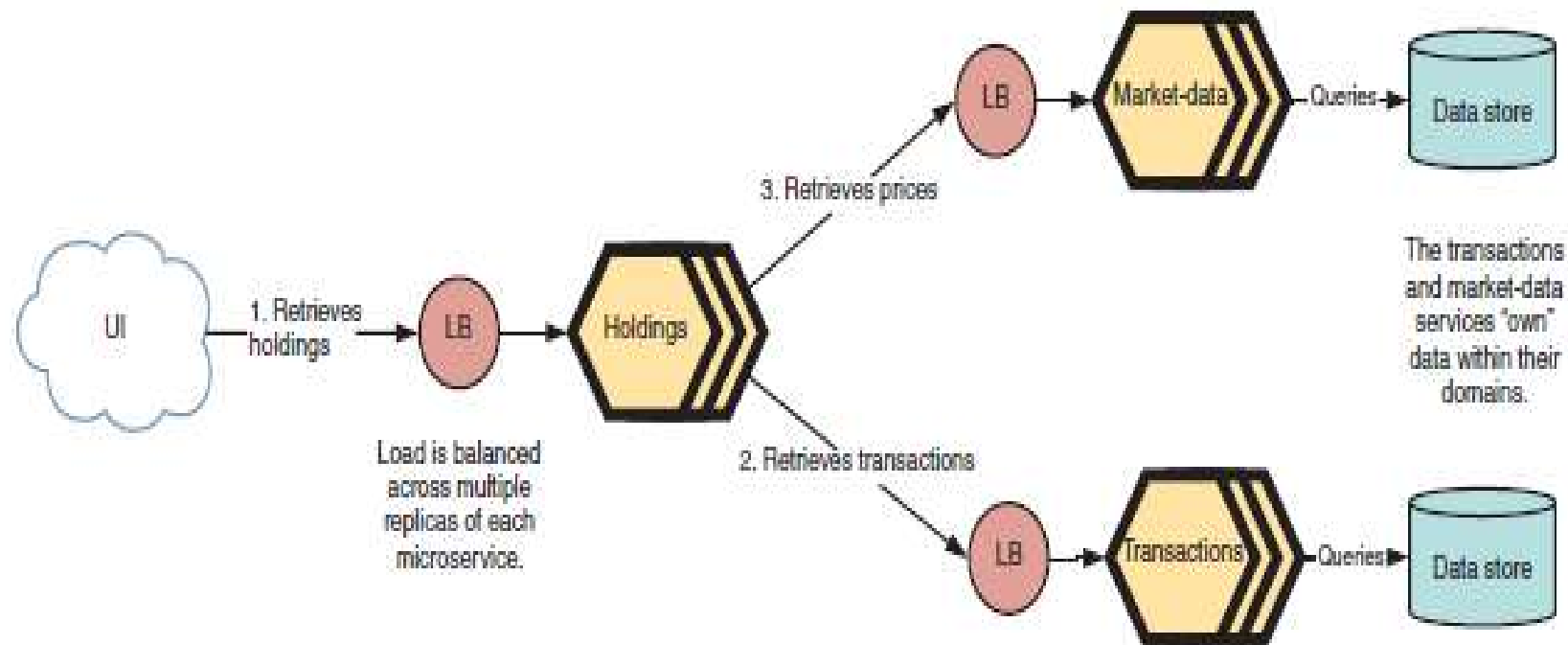
Source of failure	Description
Timeouts	Requests to services may time out, resulting in erroneous behavior.
Decommissioned or nonbackwards-compatible functionality	Design doesn't take service dependencies into account, unexpectedly changing or removing functionality.
Internal component failures	Problems with databases or caches prevent services from working correctly.
External dependencies	Services may have dependencies outside of the application that don't work correctly or as expected – for example, third-party APIs.

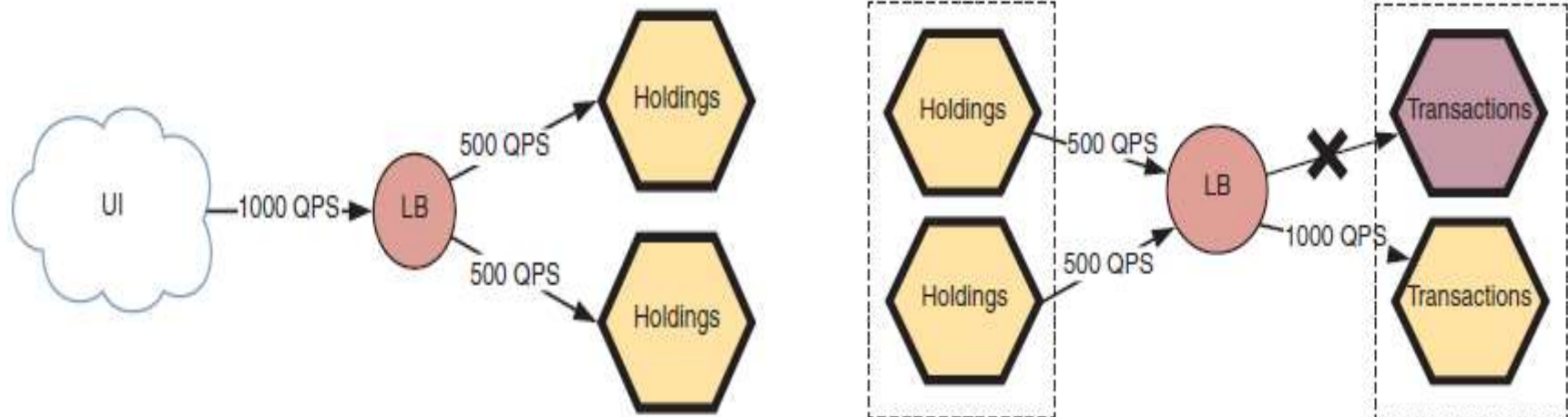
Cascading failures



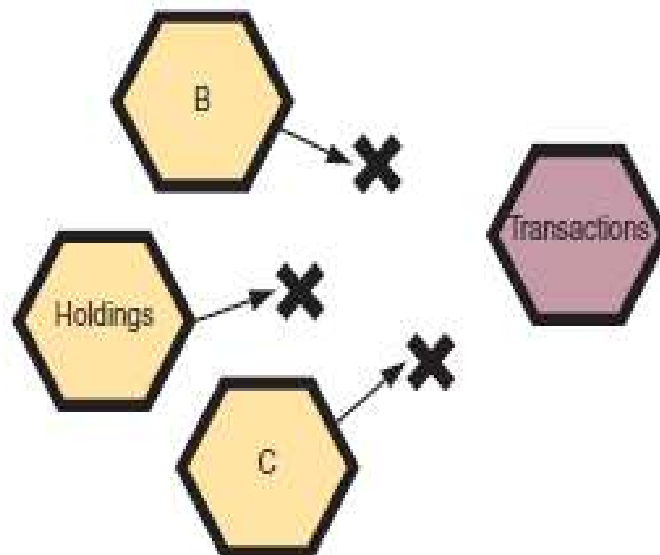
- In a microservice application, overload can cause a domino effect: failure in one service increases failure in upstream services, and in turn their upstream services. At worst, the result is widespread unavailability

Example: cascading failure

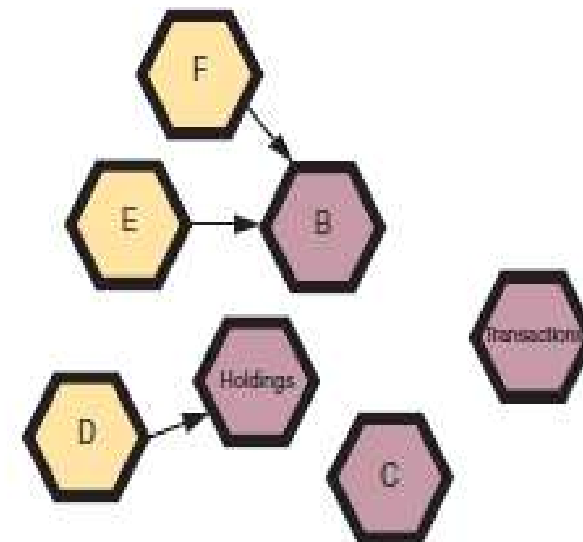




Upstream dependencies are unable to service requests that rely on transactions.



Increased failure in upstream dependencies leads to retries, repeating the cycle of failure.



Designing reliable communication



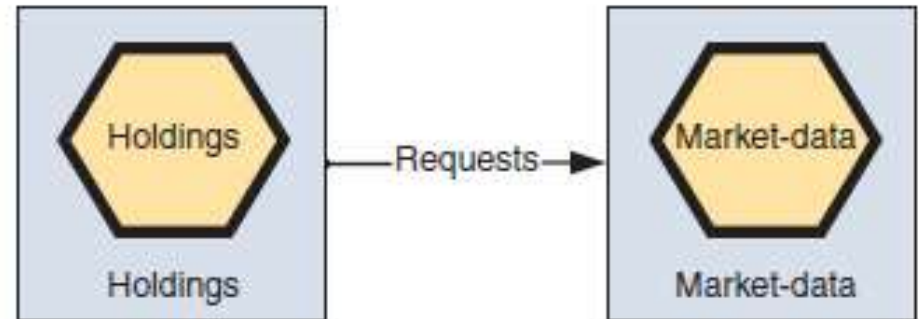
Several techniques for ensuring that services behave appropriately

- Retries
- Fallbacks, caching, and graceful degradation
- Timeouts and deadlines
- Circuit breakers
- Communication brokers

Retries



- Imagine that a call from the holdings service to retrieve prices fails, returning an error.
- From the perspective of the calling service, it's not clear yet whether this failure is isolated
- Repeating that call is likely to succeed, or systemic i.e. the next call has a high likelihood of failing.



Retries



- How can you use retries to improve your resiliency in the face of intermittent failures without contributing to wider system failure if persistent failures occur?
- First, you could use a variable time between successive retries to try to spread them out evenly and reduce the frequency of retry-based load.

Retries: few points to remember



- Always limit the total number of retries.
- Use exponential back-off with jitter to smoothly distribute retry requests and avoid compounding load.
- Consider which error conditions should trigger a retry and, therefore, which retries are unlikely to, or will never, succeed.

Asynchronous communication



- It's recommended to use asynchronous communication when microservices need to interact **without blocking execution**.

Use Cases:

- **Event-Driven Architectures:** When a service needs to notify other services about an event but does not require an immediate response.
- When you want to **prevent a failure in one service from affecting others**.
- When multiple microservices handle a high volume of requests and blocking calls can degrade performance.
- When an operation takes too long and holding an HTTP connection is impractical.
- When multiple services must consume an event without dependencies.

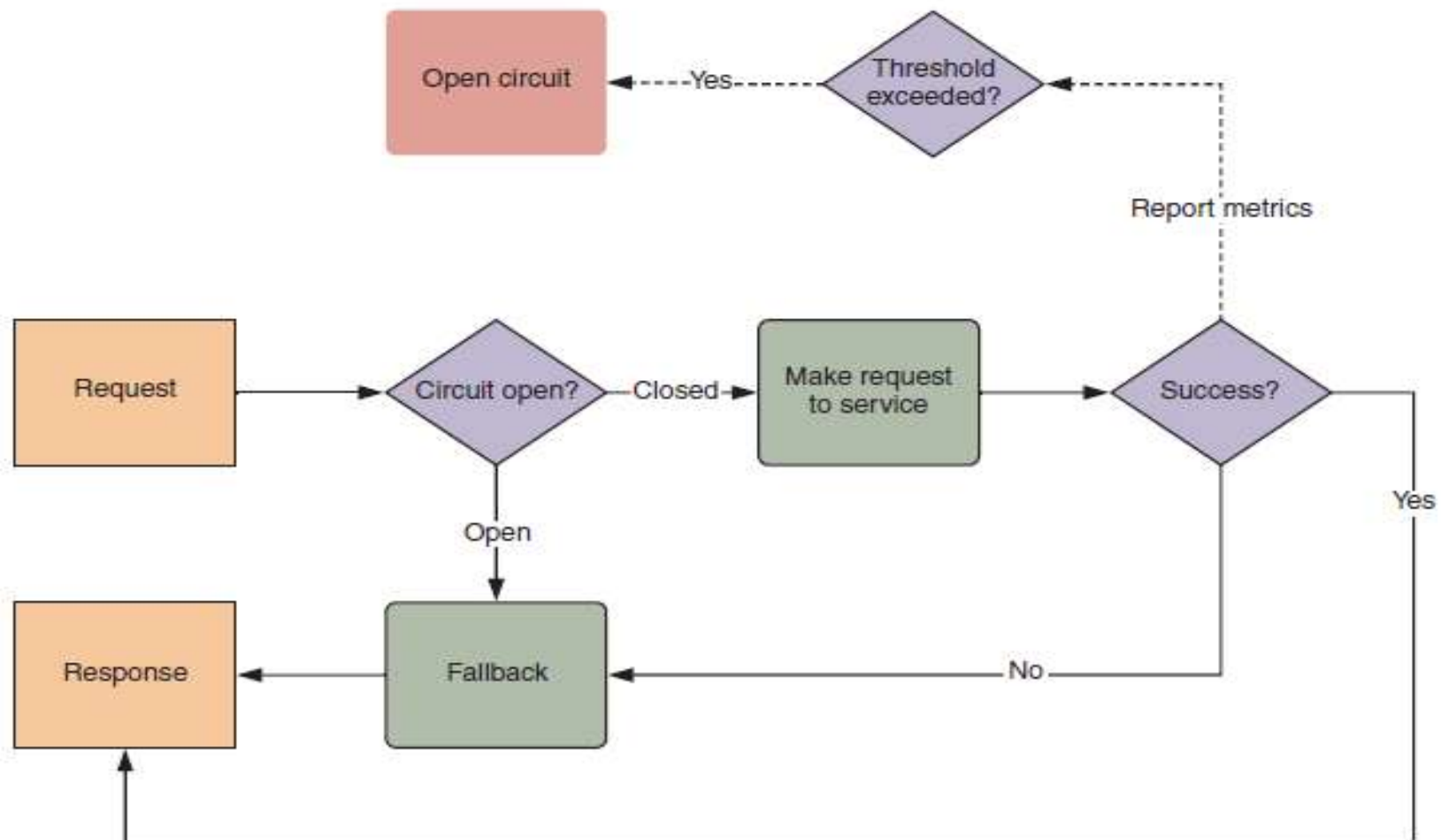
Circuit breakers



- A circuit breaker is a pattern for pausing requests made to a failing service to prevent cascading failures.

Approach

- When making a request to a service, you can track the number of times that request succeeds or fails
- In case of normal operation, we consider the circuit to be closed.
- If the number of failures seen or the rate of failures within a certain time window passes a threshold, then the circuit is opened



Why Use Circuit Breakers?



- **Prevent System Overload:** Stops excessive requests to a failing service, reducing unnecessary resource consumption.
- **Improve System Stability:** Ensures that failures in one microservice don't bring down the entire system.
- **Enable Fast Failure & Recovery:** Redirects traffic or returns fallback responses instead of waiting for a failed service.



BITS Pilani
Pilani Campus



Maximizing service reliability

Load Balancer



The load balancer plays two important roles:

- **Health Check:** Identifying which underlying instances are healthy and able to serve requests
- **Routing requests** to different underlying instances of the service

We can classify health checks based on two criteria:

- **Liveness:** check that the application has started and is running correctly.
- **Readiness:** indicates whether a service is ready to serve traffic

Rate limits



- Technique used to control the number of requests a client can make to a service within a specific time period.

Strategy	Description
Drop requests above volume	Drop consumer requests above a specified volume
Prioritize critical traffic	Drop requests to low-priority endpoints to prioritize resources for critical traffic
Drop uncommon clients	Prioritize frequent consumers of the service over infrequent users
Limit concurrent requests	Limit the overall number of requests an upstream service can make over a time period

Rate limiting in Microservices



- API Gateway-based Rate Limiting
- Middleware-based Rate Limiting
- Cloud Provider Solutions

Example Use Cases

✓ Protecting Authentication Services

Limit failed login attempts (e.g., **5 attempts per minute** to prevent brute force attacks).

✓ Ensuring Fair API Usage

Free-tier users can make **100 API calls per hour**, while premium users get **1000 API calls per hour**.

✓ Preventing Traffic Spikes

A content delivery API limits requests to **200 per second** to avoid overwhelming backend servers.

Throttling Pattern



- **Throttling** is a technique used to **control the rate at which requests are processed** to prevent system overload and ensure fair resource distribution.

Types of Throttling

- User-Based Throttling
- Server-Based Throttling
- Network-Based Throttling
- IP-Based Throttling

How to Implement Throttling?



- ◆ **API Gateway-Based Throttling:** Use **AWS API Gateway**, Kong, NGINX, or Azure API Management to throttle requests per client or user.
- ◆ **Application-Level Throttling:** Implement in code using Redis + Express Rate Limit (Node.js) or Spring Boot Bucket4j (Java).
- ◆ **Message Queue-Based Throttling:** Use **Kafka**, **RabbitMQ**, or **SQS** to control the rate at which requests are processed asynchronously.
- ◆ **Load Balancer-Based Throttling:** Use **AWS ALB**, **Nginx**, or **HAProxy** to distribute and throttle traffic before reaching backend services.

Queue-Based Load Levelling pattern

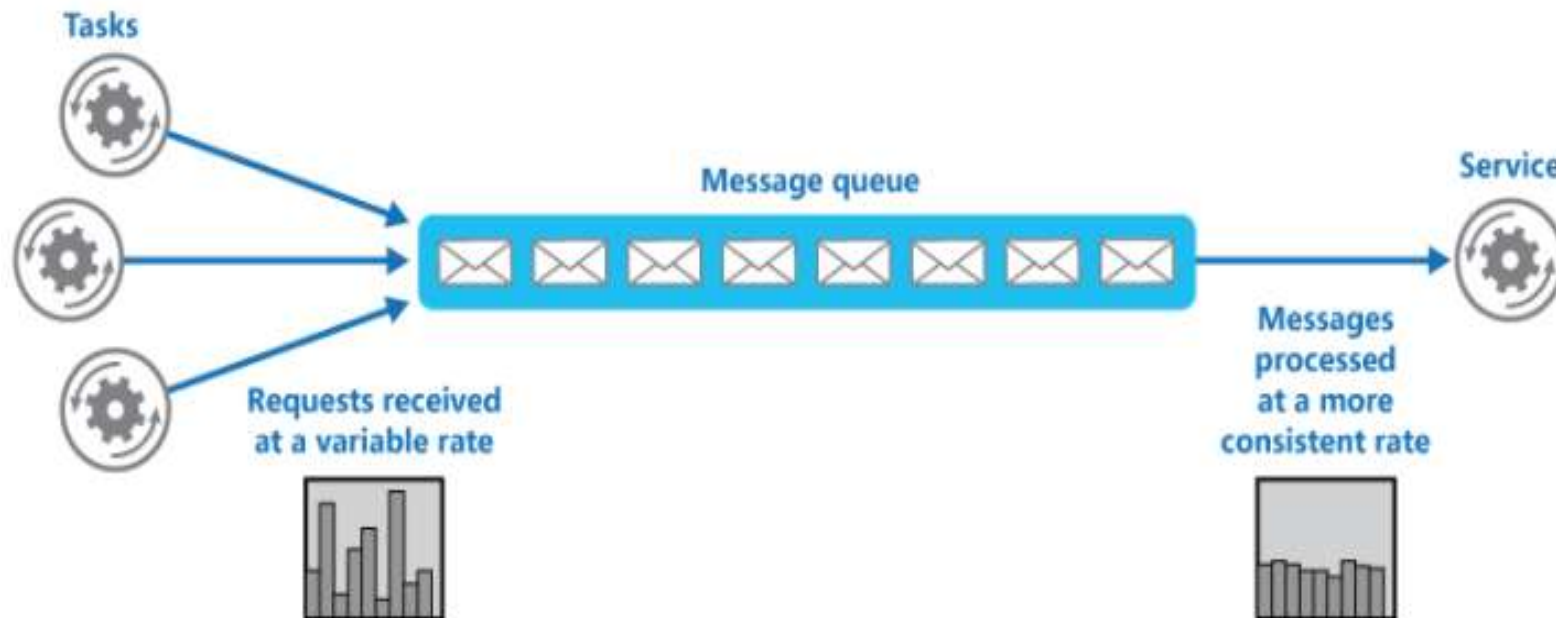


- The **Queue-Based Load Leveling Pattern** is a **scalability and resilience pattern** used in microservices to handle **high traffic spikes** by placing requests in a queue.
- This prevents services from being overwhelmed and allows them to process tasks **asynchronously at their own pace**.

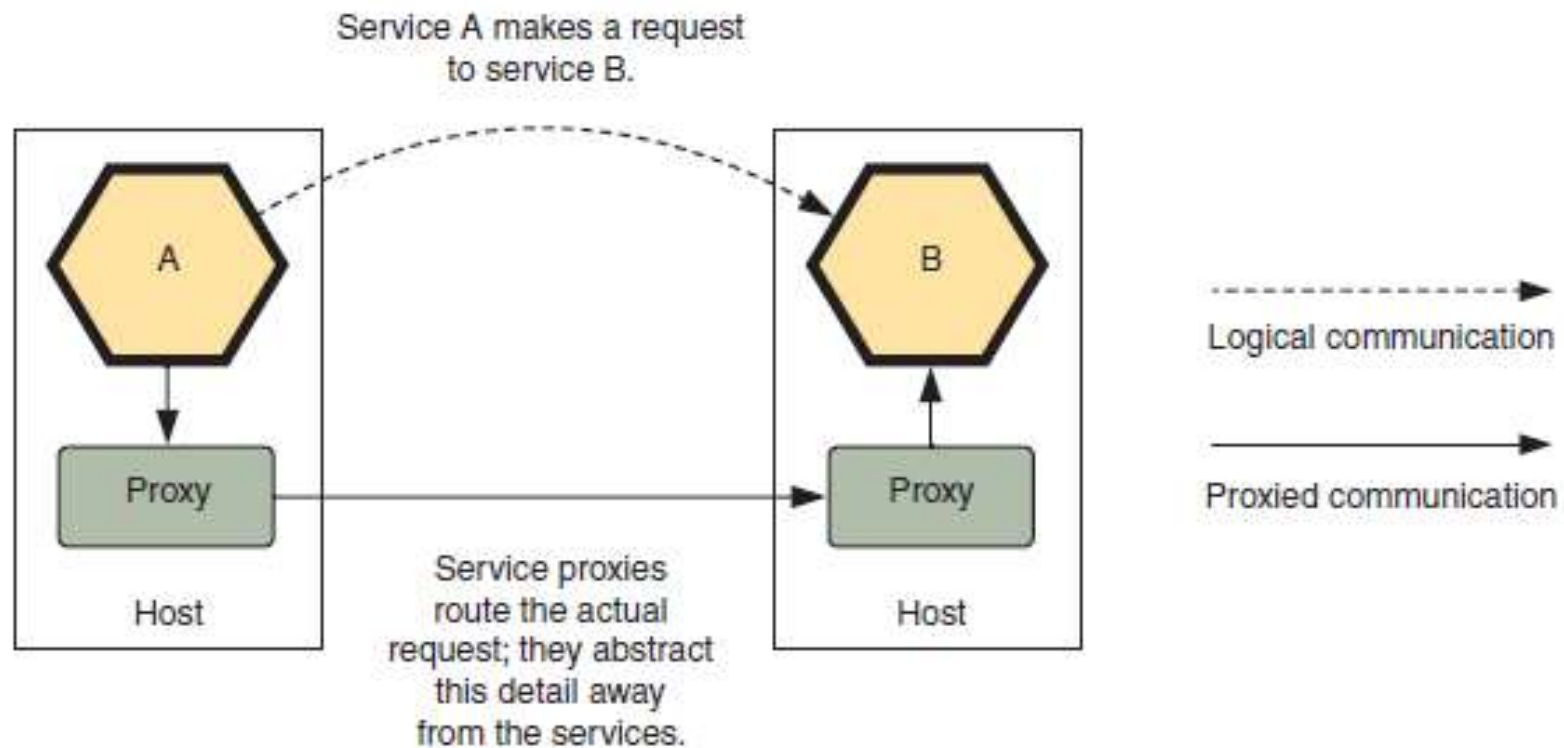
How the Pattern Works?



Solution:



Service mesh

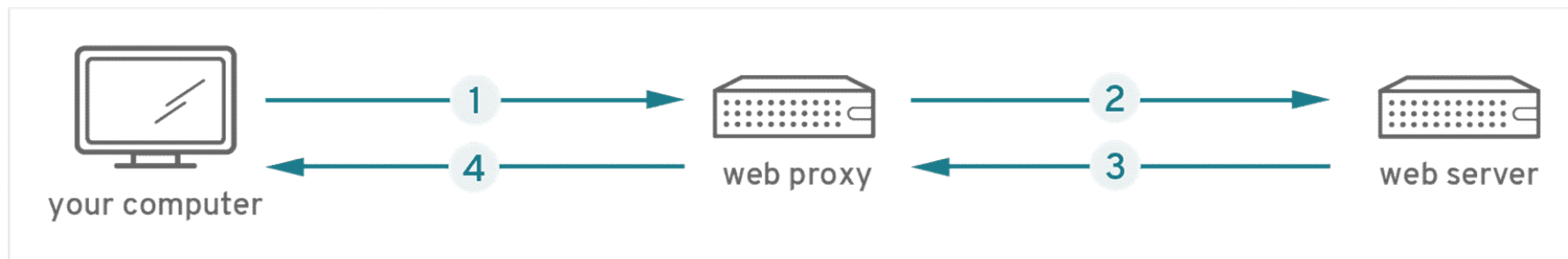


Service Mesh



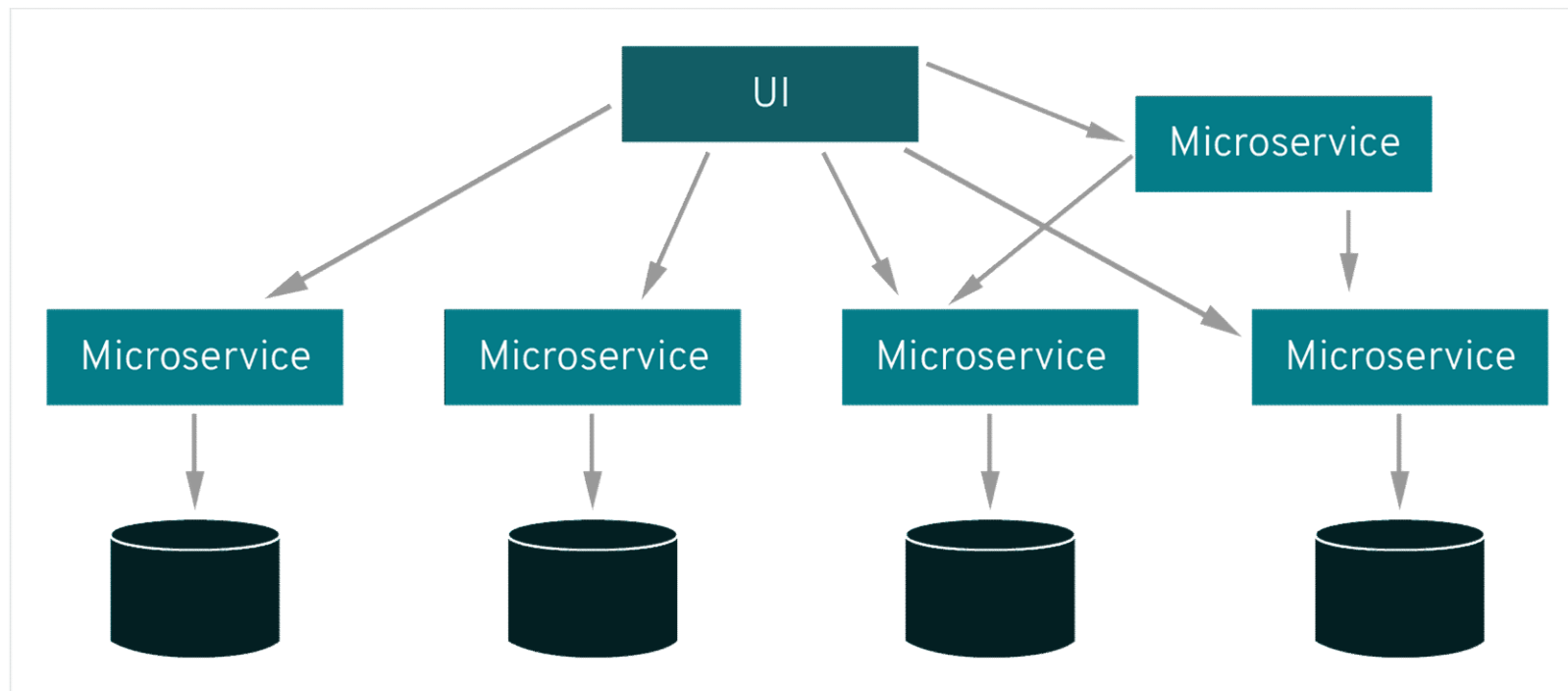
- A service mesh is a dedicated infrastructure layer designed to handle service-to-service communication within a network of microservices.
- It manages and controls how different parts of an application or service interact with each other.

How does service mesh work?

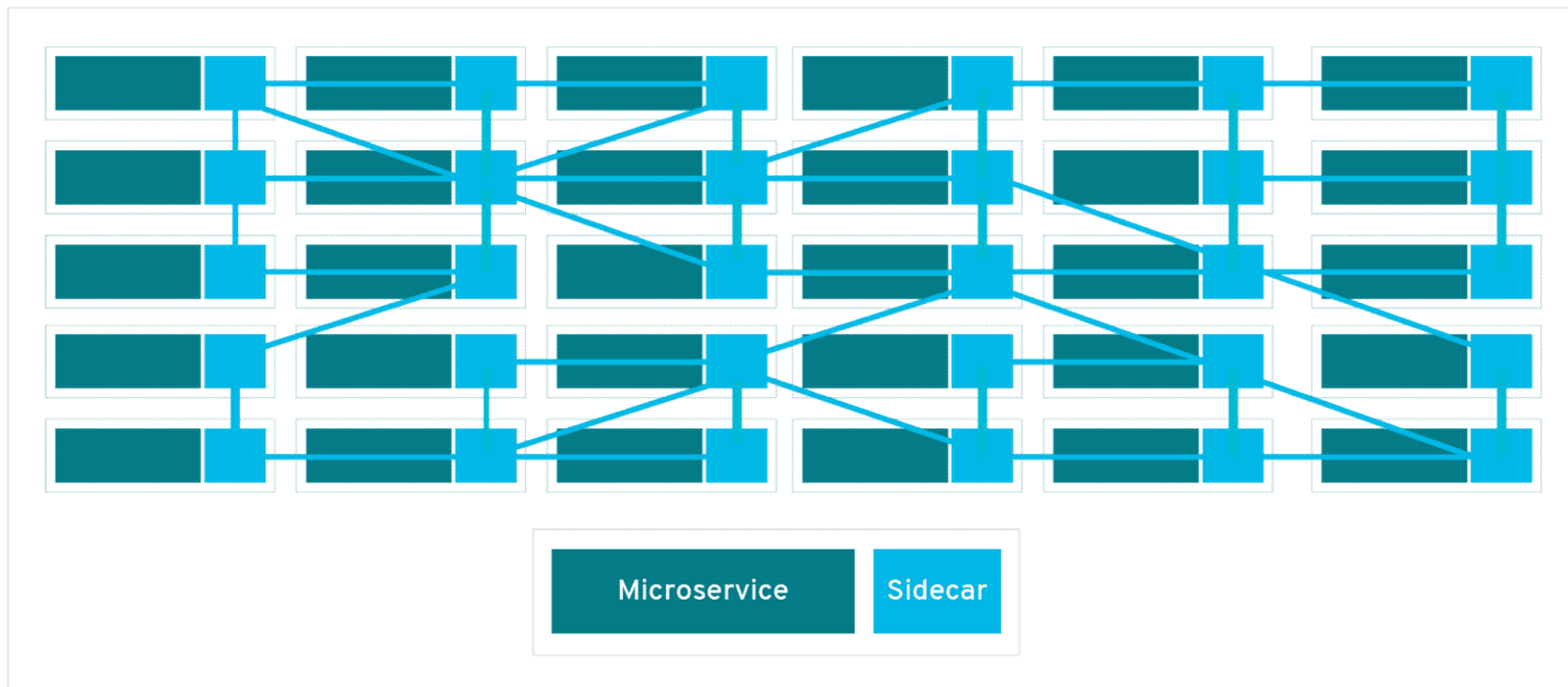


- A service mesh is built into an app as an array of network proxies.
- As your request for this page went out, it was first received by your company's web proxy
- After passing the proxy's security measure, it was sent to the server that hosts this page
- Next, this page was returned to the proxy and again checked against its security measures
- And then it was finally sent from the proxy to you.

Service Mesh and Microservices



Service Mesh and Microservices





-
- In a service mesh, requests are routed between microservices through proxies in their own infrastructure layer.
 - For this reason, individual proxies that make up a service mesh are sometimes called “sidecars,” since they run alongside each service, rather than within them.
 - Taken together, these “sidecar” proxies—decoupled from each service—form a mesh network.
 - Without a service mesh, each microservice needs to be coded with logic to govern service-to-service communication, which means developers are less focused on business goals.

Does service mesh optimize communication?



- Within a complex microservices architecture, it can become nearly impossible to locate where problems have occurred without a service mesh.
- Over time, data made visible by the service mesh can be applied to the rules for interservice communication, resulting in more efficient and reliable service requests.

Self study



- <https://cloud.google.com/architecture/rate-limiting-strategies-techniques>
- <https://aws.amazon.com/blogs/architecture/chaos-testing-with-aws-fault-injection-simulator-and-aws-codepipeline/>
- <https://learn.microsoft.com/en-us/azure/architecture/patterns/sidecar>

References



- Book: Chapter6, Microservices in Action
- <https://docs.microsoft.com/en-us/dotnet/architecture/microservices/implement-resilient-applications/partial-failure-strategies>
- <https://docs.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling>
- <https://docs.microsoft.com/en-us/azure/architecture/patterns/throttling>
- <https://www.redhat.com/en/topics/microservices/what-is-a-service-mesh>